

Text to Token: How It Works and Why It Matters

Lesson 02, Turning Text into Numeric Tokens (01 - Text to Vectors Basics)



1. What tokenization is

A neural network only does arithmetic, so it cannot read the string *"to be or not to be"*. **Tokenization** is the step that converts text into a sequence of integers (tokens) and back. The lesson builds the simplest version: **one lowercase word = one token**, using a vocabulary of unique words and two dictionaries that are exact inverses of each other.

2. The full working, step by step

Step	What the code does	Result on the 3-sentence corpus
1. Corpus	<code>text = [...]</code> holds three short quotations	3 sentences
2. Normalize	<code>' '.join(text).lower()</code> merges and lowercases so <i>Be</i> and <i>be</i> match	one lowercase string
3. Split	<code>re.split(r'\s', ...)</code> cuts at every whitespace character	<code>allwords</code> : 28 words
4. Vocabulary	<code>sorted(set(allwords))</code> removes duplicates, then orders them alphabetically	<code>vocab</code> : 20 unique words
5. Lookup tables	<code>enumerate(vocab)</code> builds <code>word2idx</code> and <code>idx2word</code>	<code>all=0</code> , <code>already=1</code> , ... <code>yourself=19</code>
6. Encode	<code>[word2idx[w] for w in allwords]</code>	<code>[0, 13, 17, 2, 6, 14, 11, ...]</code>
7. Decode	<code>[idx2word[i] for i in ids]</code>	original words, lowercase

Why sorted? A Python set has no guaranteed order, so without sorting the same text could receive different IDs on different runs. Sorting makes the mapping deterministic. **Why two dictionaries?** A dictionary looks up in one direction only; `word2idx` encodes and `idx2word` decodes, and the round trip decoder (`encoder(x)`) is the basic correctness test.

3. Code explained

```
import re
text = ['All that we are is the result of what we have thought',
        'To be or not to be that is the question',
        'Be yourself everyone is already taken']

allwords = re.split(r'\s', ' '.join(text).lower()) # 28 tokens (with repeats)
vocab     = sorted(set(allwords))                  # 20 unique, alphabetical

word2idx = {w: i for i, w in enumerate(vocab)}      # notebook uses an explicit loop
idx2word = {i: w for i, w in enumerate(vocab)}

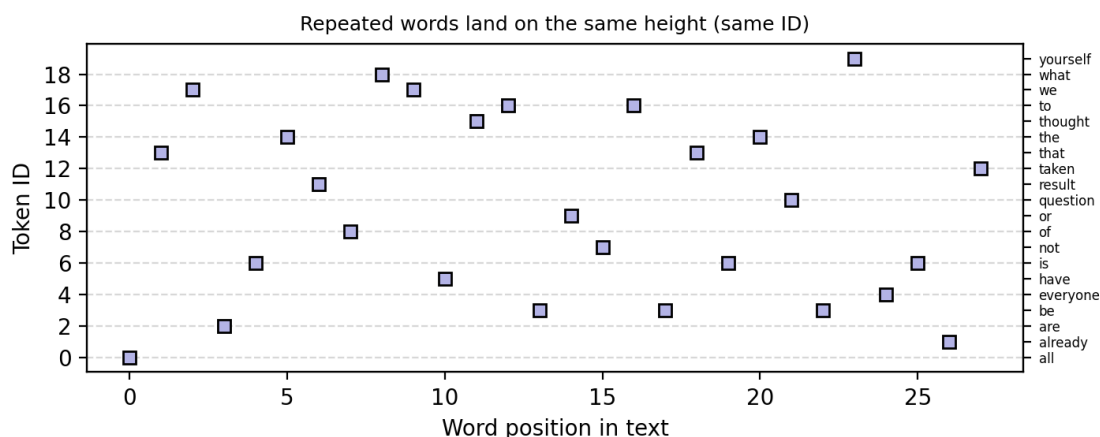
def encoder(text):
    return [word2idx[w] for w in re.split(' ', text.lower())]

def decoder(indices):
    return [idx2word[i] for i in indices]

encoder('to be')          # [16, 3]
decoder(encoder('to be')) # ['to', 'be']
```

- **Encoder:** lowercase, split on spaces, look every word up in `word2idx`. Unknown words raise a `KeyError`.
- **Decoder:** look every integer up in `idx2word`. It returns a list, so use `' '.join(...)` for a sentence.
- **Fake quotes:** `np.random.randint` picks random IDs that decode to word salad. That is a language model with no knowledge at all, which shows what training must improve.

4. Seeing the tokens



Each square is one word of the text. Same height means same word: *be* (ID 3) and *is* (ID 6) each appear three times. A text is just a sequence of integers, and repetition becomes visible structure.

5. Context around a token

The notebook finds every position of *to* with `np.where` and prints the token before and after it: `[15, 16, 3]` ("thought to be") and `[7, 16, 3]` ("not to be"). The ID of *to* is identical (16), but its neighbours differ. This is a preview of how language models learn: a token is understood by the company it keeps. Edge case: at index 0, tokens `[t-1:t+2]` becomes tokens `[-1:2]`, an empty slice.

6. Significance in an LLM

- **It is the gateway.** Every LLM stage (embeddings, attention, output layer) assumes text is already numeric, and generated numbers are decoded back to text by the same table.
- **Vocabulary size sets model size.** The input embedding table and the output layer each have one row or column per token, so a bigger vocabulary means more parameters and a larger softmax.
- **Tokenization sets sequence length.** Finer pieces mean more tokens per sentence, which means more compute and a smaller effective context window.
- **It must be consistent.** The same tokenizer and vocabulary must be used for training and inference. Change the corpus and IDs shift, so save the vocabulary with the data.
- **IDs are labels, not quantities.** The number 16 is not "bigger" than 3; it is just alphabetical position. Giving numbers meaning is the job of embeddings (lesson 03).

7. Important things to remember

Weakness of this tokenizer	Example (verified by running the code)	Real-world fix
Unknown words crash it	<code>encoder('hello')</code> raises <code>KeyError: 'hello'</code>	Special <code><UNK></code> token, or subword pieces
Punctuation sticks to words	<code>encoder('to be, or')</code> raises <code>KeyError: 'be,'</code>	Split punctuation with a regex or tokenizer rules
Fragile whitespace	trailing or double space gives an empty string, <code>KeyError: ''</code>	<code>text.split()</code> collapses whitespace
Lowercasing loses information	<i>Apple</i> and <i>apple</i> become one token	Case-sensitive or byte-level tokenizers
Huge vocabulary on real text	every word form is a separate entry	Subword methods such as Byte Pair Encoding

From toy to real: production LLMs keep the same two-table idea (token to ID and ID to token) but split text into **subwords** (for example *token* + *ization*), add special tokens for padding and end-of-text, and learn the vocabulary from a large corpus with tens of thousands of entries or more. Words are too many, characters make sequences too long, and subwords sit in the middle.

Takeaway: tokenization is reversible, deterministic text-to-integer conversion. Master this small version and the real ones, and the embeddings that follow, become straightforward.